

Основы вайб-кодинга

Как пройти путь от идеи до работающего прототипа с Qwen/GigaChat, OpenCode, GitVerse и GigaCode

1 Проблема	2 Критерии	3 План	4 Маленький diff	5 Запуск	6 Проверка	7 Commit
---------------	---------------	-----------	---------------------	-------------	---------------	-------------

Главная цель — не сгенерировать больше кода, а доказать, что команда решила конкретную проблему и понимает результат.

Пять правил

- ✓ **Сначала критерии, потом код.** Нужно заранее определить, что считается готовым.
- ✓ **Одна итерация — одна небольшая задача.** Маленький diff легче понять, проверить и отменить.
- ✓ **Ответ AI — не доказательство.** Важный diff полезно проверить моделью другого производителя, а затем подтвердить тестом и ``git diff``.
- ✓ **Состояние хранится в проекте.** ``PLAN.md``, ``TASKS.md`` и файлы выполнения позволяют продолжить после перезапуска.
- ✓ **Минимум разрешений и доверия.** Проверяйте команды, skills, MCP, зависимости и секреты.

ИИ помогает • уточнять требования и сокращать MVP; • читать репозиторий и предлагать план; • создавать небольшой код и тесты; • объяснять diff и возможные ошибки.	Человек отвечает • за постановку задачи и ограничения; • за разрешения и безопасность; • за проверку фактических изменений; • за итоговое решение и объяснение.
---	--

ПЛАНИРОВАНИЕ ДО КОДА

Qwen Chat, GigaChat и GitVerse

Продуктовое описание и черновой PLAN.md готовим в отдельном чате, сохраняем в GitVerse и только затем запускаем OpenCode

Qwen Chat

- Авторизуйтесь: без входа гостевой лимит заканчивается быстрее.
- Создайте Workspace проекта или команды.
- Загрузите постановку, правила, заметки и разрешённые примеры.
- Попросите PRODUCT.md и черновой PLAN.md на русском.
- В PLAN.md включите этапы, зависимости, риски и тестирование.
- Не загружайте API-ключи, .env и персональные данные.

GigaChat — альтернатива

- Используйте, если Qwen недоступен или лимит закончился.
- Попросите те же PRODUCT.md и черновой PLAN.md.
- Согласуйте план до запуска кодового агента.
- Сохраните результат в файлы и GitVerse.
- Не путайте GigaChat с GigaCode — помощником для кода в IDE.



В Qwen Chat или GigaChat подготовьте PRODUCT.md и черновой PLAN.md. Это не расходует токены модели-исполнителя: OpenCode сверяет готовый план с репозиторием и занимается реализацией, тестами и исправлениями.

Как устроен Qwen Workspace

WORKSPACE: ПРОЕКТ

- постановка.pdf — исходная задача
- правила.md — ограничения
- заметки.md — решения и вопросы
- примеры.csv — данные или примеры
- PRODUCT.md — описание продукта
- PLAN.md — план реализации и тестирования

Git и GitVerse

- Git хранит локальную историю изменений.
- GitVerse — рекомендуемый удалённый репозиторий курса.
- Создайте репозиторий до запуска агента.
- Делайте commit после каждого проверенного шага.

Минимальная публикация

- ``git init``
- ``git add .``
- ``git commit -m "init: каркас проекта"``
- ``git branch -M main``
- ``git remote add origin <URL GitVerse>``
- ``git push -u origin main``



Qwen: авторизуйтесь заранее — гостевой лимит быстро заканчивается. Условия Qwen/GigaChat могут измениться. Не коммитьте .env, токены и ключи в GitVerse.

УСТАНОВКА И ЗАПУСК

Установка, запуск и интерфейс

Короткий путь: установить CLI → открыть каталог проекта → подключить модель → начать сессию

УСТАНОВИТЬ (LINUX / macOS / WSL)
`curl -fsSL https://opencode.ai/install | bash`

АЛЬТЕРНАТИВА С NODE.JS
`npm install -g opencode-ai`

ПРОВЕРИТЬ И ЗАПУСТИТЬ
`opencode --version`
`cd /путь/к/проекту`
`opencode`

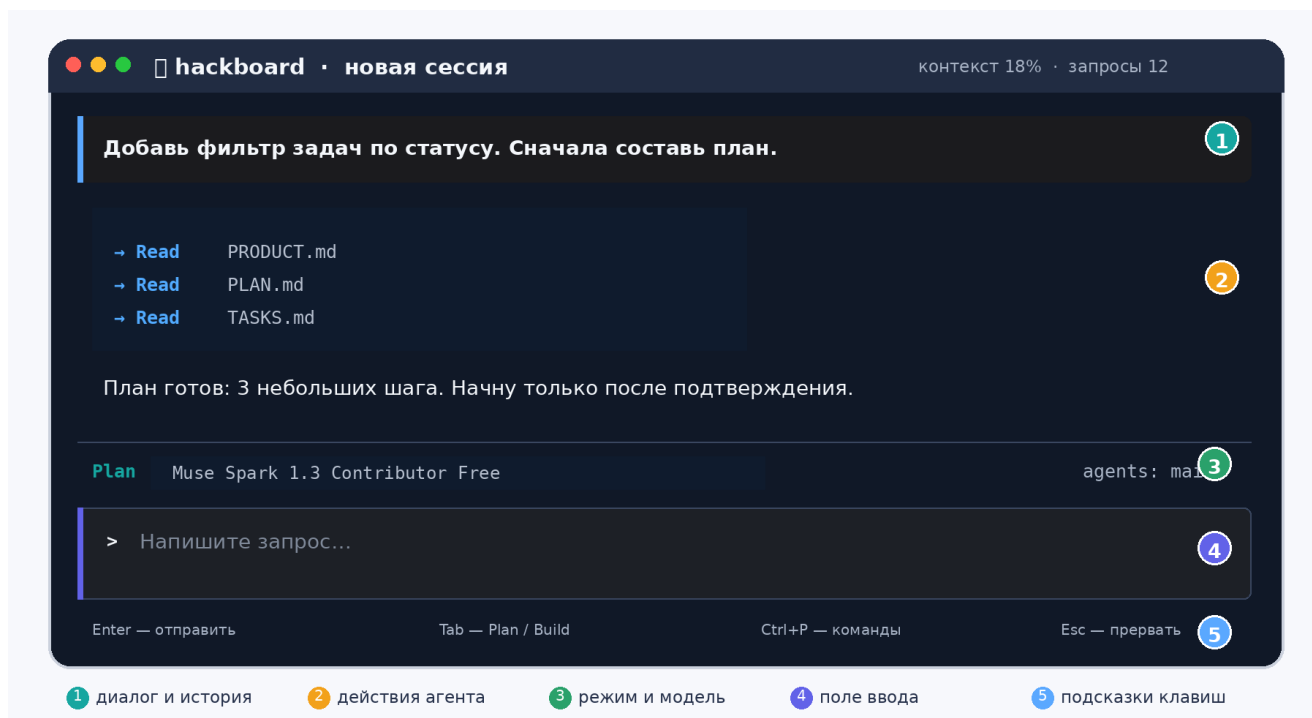
Первый вход

- `/connect` — подключить провайдера;
- `/models` — выбрать доступную модель;
- `/init` — изучить проект и создать AGENTS.md;
- `@файл` — добавить файл в контекст;
- `Tab` — переключить PLAN и BUILD.

Как начать безопасно

- сначала попросить объяснить структуру проекта;
- запросить план и запретить менять файлы;
- давать только одну небольшую задачу;
- после шага смотреть `git diff`;
- `Esc` — прервать ненужный длинный цикл.

Как выглядит интерфейс



!

Windows: рекомендуется WSL. Команда ``curl | bash`` запускает код из сети — используйте только официальный адрес `opencode.ai`. Если команда ``opencode`` не найдена, перезапустите терминал. Вид интерфейса зависит от версии и темы.

БЫСТРЫЙ СТАРТ

Базовые знания и OpenCode

Не нужно помнить весь синтаксис, но нужно уметь проверить работу агента

Минимальные навыки

- найти точку входа, исходный код и тесты;
- запустить и остановить приложение;
- описать ошибку: шаги, ожидание, факт;
- объяснить назначение функции и данных;
- **посмотреть Git status, diff и commits; создать общий репозиторий в GitVerse.**

Пять действий OpenCode

- ``opencode`` — запустить агент в каталоге;
- ``/init`` — изучить проект и создать AGENTS.md;
- ``/models`` — выбрать подключённую модель;
- ``@файл`` — явно добавить файл в контекст;
- ``Tab`` — переключить Plan и Build.

МИНИМАЛЬНЫЕ КОМАНДЫ

```
pwd
ls
git status
git diff
git log --oneline -5
npm test
npm run check
```

Правильный цикл в OpenCode

Режим	Что делает агент	Фраза-ограничитель
PLAN	Читает проект, называет файлы и риски.	«Пока ничего не изменяй».
BUILD	Реализует один согласованный шаг.	«Не переходи к следующей задаче».
VERIFY	Показывает diff, запускает проверку, обновляет статус.	«Покажи доказательства».
!		Не просите «сделай весь проект». Просите «реализуй один проверяемый шаг и остановись».

AGENTS.md и иерархические инструкции

Корневой файл хранит стабильные правила и направляет агента к деталям

Что включить

- назначение и карта каталогов;
- команды запуска и проверки;
- стабильные ограничения;
- правила обновления TASKS.md;
- делегирование субагентам;
- язык документов и безопасность.

Что вынести

- текущие временные задачи;
- весь README и учебники;
- описание каждой функции;
- историю переписки;
- все skills и правила модулей целиком;
- устаревающие решения.

!

AGENTS.md должен быть коротким. Практический ориентир — десятки строк, а не сотни. Это рекомендация, а не технический лимит.

AGENTS.md

общие правила и условия чтения деталей

src/AGENTS.md

правила исходного кода

tests/AGENTS.md

правила тестирования

docs/rules/*.md

специализированные инструкции по задаче

Минимальный пример

```
AGENTS.MD
# Правила проекта

- Все документы, планы, статусы и отчёты пишите на русском языке.
- Имена файлов, команд, API и идентификаторов кода не переводите.
- Перед работой прочитайте PRODUCT.md, PLAN.md и TASKS.md.
- Проверьте git status и git diff.
- Создайте уникальный файл выполнения и продолжайте с поля «Точный следующий шаг».
- Выполняйте одну небольшую задачу за итерацию.
- Не добавляйте зависимости и не меняйте несвязанные файлы без одобрения.
- Перед изменением src/ прочитайте src/AGENTS.md.
- Не загружайте все модульные инструкции заранее.
```

!

Ссылки внутри AGENTS.md не раскрываются автоматически. Явно напишите: когда и какой файл агент должен прочитать.

ПЛАН, ЗАДАЧИ И ЗАПУСКИ

PLAN.md, TASKS.md и файлы выполнения

PLAN.md хранит стратегию реализации и тестирования; TASKS.md — конкретные задачи и состояние; каждая попытка получает отдельный файл с UTC-меткой

PLAN.md — план реализации и тестирования
 Этап 1: модель данных
 Реализация: определить Task и формат хранения.
 Проверка: unit-тесты serialize/load.
 Критерий: status сохраняется после refresh.

TASKS.md — конкретная задача
 T-003 | IN_PROGRESS | storage | 2/4
 Scope: src/storage.js, tests/storage.test.js
 Решение: load() читает сохранённый status.
 Текущая попытка: docs/agent/tasks/T-003/
 20260903T101530Z_storage_a01.md
 Next: исправить load() и запустить npm test.

PLAN.md — стратегия

- этапы и зависимости;
- архитектурные ограничения;
- стратегия unit / integration / manual тестов;
- риски, критерии и откат;
- меняется только при пересмотре стратегии.

TASKS.md — конкретные задачи

- цель, score и разрешённые файлы;
- технические детали реализации;
- владелец, критерии и прогресс;
- блокер и точный следующий шаг;
- ссылка на текущий файл выполнения.



При начале каждой попытки создавайте новый файл выполнения. Субагент пишет только в свой уникальный файл; центральный TASKS.md обновляет один координатор после проверки.

Продолжение после перезапуска

```
СТРУКТУРА БЕЗ КОЛЛИЗИЙ
project/
├── PRODUCT.md
├── PLAN.md
├── TASKS.md
├── docs/agent/tasks/
│   ├── T-001/
│   │   └── 20260903T091200Z_main_a01.md
│   ├── T-003/
│   │   ├── 20260903T101530Z_storage_a01.md
│   │   └── 20260903T104212Z_reviewer_a02.md
```

Имя: YYYYMMDDTHHMMSSZ_<OWNER>_aNN.md
 UTC + владелец + номер попытки предотвращают коллизии. Новая сессия или субагент создаёт новый файл; если имя занято, увеличьте aNN. Старые файлы не перезаписываются.

Внутри: цель, score, критерии, файлы, команды, результаты, блокеры, next step, HANDOFF и ссылка на предыдущий запуск.

После перезапуска: AGENTS → PRODUCT → PLAN → TASKS → последний файл выполнения → git diff → тесты. Создайте новый файл выполнения и продолжите с next step.

Субагенты, MCP и LSP

Передача работы должна переживать завершение отдельной сессии

Основной агент назначает T-003 и владельца в TASKS.md	Исполнитель создаёт свой уникальный файл выполнения	HANDOFF фиксирует результат и следующий шаг	Основной агент проверяет diff и тесты	TASKS.md получает проверенный статус
---	---	---	---------------------------------------	--------------------------------------

- Основной агент является единственным владельцем центрального `TASKS.md` и переносит в него только проверенные итоги файлов выполнения.
- Субагент создаёт и обновляет только свой уникальный файл выполнения и разрешённые исходные файлы.
- В конце файл выполнения содержит `SUBAGENT\_HANDOFF`: критерии, файлы, команды, результаты, блокиеры и точный следующий шаг.
- **Основной агент** проверяет фактический diff и только потом обновляет общий статус.

MCP • подключает внешние инструменты и данные; • `websearch` — актуальный веб-поиск; • `Context7` — документация библиотек; • `grep.app` — примеры в открытом коде; • `CodeGraph` — граф файлов, символов, вызовов и зависимостей.	LSP • понимает структуру локального кода; • находит определения и ссылки; • показывает типы и диагностику; • помогает безопаснее переименовывать символы; • не заменяет тесты бизнес-логики.
--	--



Подключайте MCP и LSP только по необходимости. Для независимого review по возможности используйте модель другого производителя: она реже повторяет те же типичные ошибки модели-исполнителя. Любой вывод всё равно проверяйте по Git diff и тестам.

Правило использования внешней информации

Найти → проверить источник и версию → применить минимально → протестировать

Skill — это не только инструкция

Он может включать исполняемый код, поэтому относится к программным зависимостям

ВОЗМОЖНАЯ СТРУКТУРА

```
my-skill/
├── SKILL.md      инструкции и метаданные
├── scripts/      исполняемый код и shell-скрипты
├── references/   справочные материалы
├── assets/       шаблоны и ресурсы
└── ...
```

Что может сделать код

- читать и изменять файлы проекта;
- устанавливать пакеты и запускать команды;
- обращаться в интернет;
- читать переменные окружения и токены;
- изменять Git, удалять или отправлять данные.

Что проверить

- источник, автора, лицензию и дату;
- SKILL.md и все scripts;
- shell-команды и сетевые запросы;
- доступ к файлам, env и секретам;
- минимально необходимые разрешения;
- первый запуск в тестовом репозитории.

!

Относитесь к чужому skill как к чужой программе, которую вы собираетесь запустить на своём компьютере.

Как искать skills

- Сначала назовите повторяющуюся задачу, которую skill должен решать.
- Попросите чат-модель найти не более пяти реальных репозиторий и привести ссылки.
- Потребуйте лицензию, дату обновления, список scripts, разрешения, сетевой доступ и риски.
- Проверьте, что ссылки существуют и содержимое соответствует описанию.
- Выберите один непересекающийся вариант; не устанавливайте коллекцию «на все случаи жизни».

НЕ ЗАПУСКАТЬ БЕЗ ПОНИМАНИЯ И ПОДТВЕРЖДЕНИЯ

Красные флаги:

```
curl ... | bash      sudo ...      rm -rf ...
printenv             cat ~/.ssh/*  git push --force
```

✓

Ответ чат-модели — предварительный аудит, а не сертификат безопасности. Окончательное решение принимает человек.

БЕСПЛАТНЫЕ МОДЕЛИ OPENCODE ZEN

Бесплатные модели OpenCode Zen

Актуальный список: /models. Рекомендуемый старт — Muse Spark 1.3 Contributor Free.

Модель	Когда использовать	Статус для курса
Muse Spark 1.3 Contributor Free	Основное кодирование и отладка	Рекомендуемый старт
Muse Spark 1.2 Contributor Free	Резерв предыдущего поколения	Запасной вариант
MiMo-V2.5 Free	Повседневные агентные задачи	Универсальный резерв
Nemotron 3.5 Lightning Free	Быстрые шаги и проверки	Скоростной резерв
Nemotron 3 Ultra Free	Сложный анализ и координация	Для тяжёлых этапов
Ling 3.0 Flash Fin Free	Узкая финансовая специализация	Не основной вариант
Big Pickle	Нет прозрачного описания	Не стандарт курса



Проверено 03.09.2026. Все free-модели OpenCode Zen доступны временно: список, скорость и ограничения могут измениться. Перед занятием проверяйте `/models`. У endpoint различаются правила обработки данных — не отправляйте закрытый код, персональные данные, токены и секреты.

Выбор модели в OpenCode Zen

ПОДКЛЮЧЕНИЕ

/connect → OpenCode Zen

/models → Muse Spark 1.3 Contributor Free

ЕСЛИ РЕЗУЛЬТАТ СЛАБЫЙ

Muse Spark 1.2 Contributor Free → резерв предыдущего поколения

MiMo-V2.5 Free → универсальный резерв

Nemotron 3.5 Lightning → быстрые небольшие шаги

Nemotron 3 Ultra → сложный анализ

ЭКОНОМИЯ

Qwen Chat / GigaChat → PRODUCT.md и черновой PLAN.md

OpenCode Plan → сверка PLAN.md с репозиторием

OpenCode Build → одна задача из TASKS.md

Проверка → diff, тесты, файл выполнения и commit

- Перед занятием: `/connect` → OpenCode Zen → `/models`; убедитесь, что выбранная free-модель доступна.
- Экономия: PRODUCT.md и черновой PLAN.md готовьте в Qwen Chat / GigaChat; в OpenCode выполняйте одну задачу из TASKS.md за раз.
- Безопасность: не отправляйте секреты, персональные данные и закрытый код; у endpoint различаются правила использования данных.
- Если Muse Spark 1.3 недоступна или слаба на конкретной задаче, переключитесь на MiMo-V2.5 Free; Muse Spark 1.2 оставьте запасным вариантом.



Muse Spark 1.3 Contributor Free — рекомендуемый старт для учебного кодирования; MiMo-V2.5 Free — универсальный резерв. Для review полезно переключиться на модель другого производителя, например Muse → MiMo или Nemotron: разные семейства имеют разные слепые зоны. Это практическая схема, а не абсолютный рейтинг. Бесплатность временная; не отправляйте секреты и закрытый код.

Промпты, роли и сдача проекта

Команда работает над одним основным пользовательским сценарием

Qwen Chat / GigaChat «Сначала задай до пяти вопросов. Затем создай PRODUCT.md на русском. После этого подготовь черновой PLAN.md: этапы реализации, зависимости, риски и стратегия тестирования. Код не пиши».	OpenCode «Сверь PRODUCT.md и PLAN.md с репозиторием. Создай в TASKS.md конкретные задачи с деталями реализации. Перед выполнением текущей задачи создай docs/agent/tasks/<ID>/<UTC>_<OWNER>_aNN.md. Реализуй только одну задачу, запусти проверки и остановись».	GigaCode / другая модель «Проверь только незакоммиченный diff. Ищи несоответствие требованиям, лишние изменения, ошибки и отсутствующие тесты. Сначала код не меняй». По возможности выбирайте модель другого производителя.
--	--	--

Роли команды из трёх человек

- **Product owner:** поддерживает PRODUCT.md и границы MVP.
- **Agent operator:** формулирует задания OpenCode и следит за TASKS.md.
- **Reviewer:** смотрит diff, запускает проверки и работает с GigaCode.
- **Ротация:** каждый участник должен попробовать все три роли и понимать весь процесс.

Что сдаём • PRODUCT.md, PLAN.md, TASKS.md, README.md; • AI_NOTES.md с принятыми и отклонёнными советами; • репозиторий GitVerse с осмысленной историей commits; • работающий основной сценарий; • трёхминутную демонстрацию.	Что оценивается • 25% — проблема и реалистичный MVP; • 35% — работающий сценарий; • 20% — diff, тесты и обработка ошибок; • 20% — объяснение решений командой; • не оценивается объём кода и число моделей.
---	--



На защите каждый участник объясняет хотя бы одно изменение и один отказ от предложения ИИ.

Официальные источники: OpenCode Zen · OpenCode MCP/LSP · CodeGraphContext · Qwen Chat / Workspace · GigaChat · GitVerse · GigaCode · Agent Skills.